

Fast-Match Admin Panel

Documentation

OVERVIEW

1. Introduction

Fast-Match Admin Panel is a comprehensive management dashboard designed to provide administrative control and insights for the Fast-Match platform. Built with **React 19** and **Vite**, it offers a fast, responsive, and robust user interface.

The application allows administrators to manage users, monitor platform activity, analyze metrics, and oversee daily operations with ease. By utilizing modern web development tools like **TailwindCSS** for styling, **React Query** for data synchronization, and **Recharts** for data visualization, the admin panel delivers a seamless experience.

Table of Contents

1. Introduction
2. Technology Stack
3. System Architecture
4. Core Modules & Features
5. Routing & Navigation
6. API Integration Layer
7. Development Setup

2. Technology Stack

The dashboard leverages a powerful set of libraries and frameworks:

Core Framework

- **React 19:** Leveraging the latest React features for efficient rendering and state management.
- **Vite 6:** Ultra-fast frontend tooling for development and optimized production builds.
- **React Router Dom v7:** Handling multi-page navigation and protected administrative routes.

Data Management & Networking

- **React Query (@tanstack/react-query):** Used for robust server-state management, automated caching, and background data fetching.
- **Axios:** Standardized HTTP client for executing API requests.

UI & Design System

- **TailwindCSS 4:** Utility-first CSS framework for building highly customizable and responsive designs.
- **Lucide React:** Providing clean, consistent vector icons throughout the interface.
- **Recharts:** Rendering dynamic and interactive charts for the Analytics and Dashboard screens.
- **Framer Motion:** Adding subtle micro-animations and transitions to enhance the user experience.

Forms & Validation

- **Formik:** Managing complex form states, especially in authentication and profile editing flows.
- **Yup:** Schema-based validation to ensure reliable data entry.

ARCHITECTURE

3. System Architecture

The project is organized to promote modularity, reusability, and scalability.

```
src/
├── components/           # Reusable UI components (buttons, modals, tables)
│   └── common/          # Shared components across multiple screens
├── main/                # Primary application screens and layouts
│   ├── auth/            # Authentication views (Login, Reset Password)
│   ├── layout/          # Dashboard shell, Sidebar, Topbar
│   └── screens/          # Feature screens (Dashboard, Users, Analytics)
├── reactQuery/           # Data fetching layer
│   ├── api/             # Axios API definitions and endpoints
│   └── hooks/            # Custom React Query hooks (useQuery, useMutation)
├── routes/              # Route definitions and access control
└── utils/               # Helper functions and global utilities
```

FEATURES

4. Core Modules & Features

4.1 Authentication

A secure gateway to the admin panel. Includes Login, Forgot Password, OTP Verification, and Password Reset flows.

4.2 Dashboard Overview

Provides a high-level summary of the platform's health. Displays key performance indicators (KPIs) and summarized charts to give administrators an instant snapshot of system status.

4.3 User Management

A comprehensive interface to view, filter, and manage platform users. Administrators can review user details, update statuses, and perform necessary administrative actions.

4.4 Activity Monitoring

Allows admins to track platform usage in real-time. This module displays active user sessions and a detailed log of important platform activities, aiding in security and troubleshooting.

4.5 Analytics

Deep-dive data visualization. Uses Recharts to render complex datasets, helping administrators identify trends, user growth, and engagement metrics.

4.6 Profile Management

Administrators can manage their own account details, update their profile information, and securely change their passwords.

5. Routing & Navigation

Routing is centralized in `src/routes/index.jsx`, separating public and private domains:

- Auth Routes:** `/login`, `/forgot-password`, `/reset-password`, `/email-verify`. These routes are accessible to unauthenticated users.
- Protected Routes:** Wrapped within a main dashboard layout. Access requires a valid authentication token. Includes `/dashboard`, `/users`, `/analytics`, etc.

API LAYER

6. API Integration Layer

All backend communications are abstracted into the `reactQuery/api` directory. The endpoints are standardized under the `/admin` prefix.

MODULE	ENDPOINT	METHOD	DESCRIPTION
Auth	<code>admin/login</code>	POST	Authenticates an administrator and returns a token.
Auth	<code>admin/logout</code>	POST	Terminates the current administrative session.
Auth	<code>admin/forgot-password</code>	POST	Initiates password recovery via email.
Auth	<code>admin/verify-otp</code>	POST	Verifies the one-time password sent to the administrator's email.

MODULE	ENDPOINT	METHOD	DESCRIPTION
Auth	admin/reset-password	POST	Sets a new password for the administrator account.
Dashboard	admin/dashboard-stats	GET	Fetches KPI data for the overview screen.
Analytics	admin/analytics	GET	Retrieves detailed charting data for analysis.
Users	admin/users	GET	Retrieves a paginated list of platform users.
Activity	admin/active-sessions	GET	Lists currently active user sessions.
Activity	admin/activity-logs	GET	Retrieves a historical log of platform activities.
Profile	admin/get-profile	GET	Fetches the profile details of the currently logged-in admin.
Profile	admin/update-profile	POST	Updates the logged-in admin's profile information.
Profile	admin/change-password	POST	Changes the logged-in admin's password securely.

DEV SETUP

7. Development Setup

To run the Fast-Match Admin Panel locally:

```
# 1. Install dependencies
npm install

# 2. Configure Environment Variables
# Create a .env file and set the backend API URL
VITE_API_URL=http://your-backend-api-url.com/api/v1

# 3. Start the development server
npm run dev

# 4. Build for production
npm run build
```

Note: Ensure that your development environment is running Node.js (v18+ recommended) for optimal compatibility with Vite and React 19.